

Farm Animals App using Classes and Objects

Experiment No: 02 Date: _____

■ Objective / Aim

To design and implement a Farm Animals App in Java using classes and objects, demonstrating object-oriented programming concepts such as inheritance, method overriding, encapsulation, and polymorphism through different types of farm animals.

■ System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

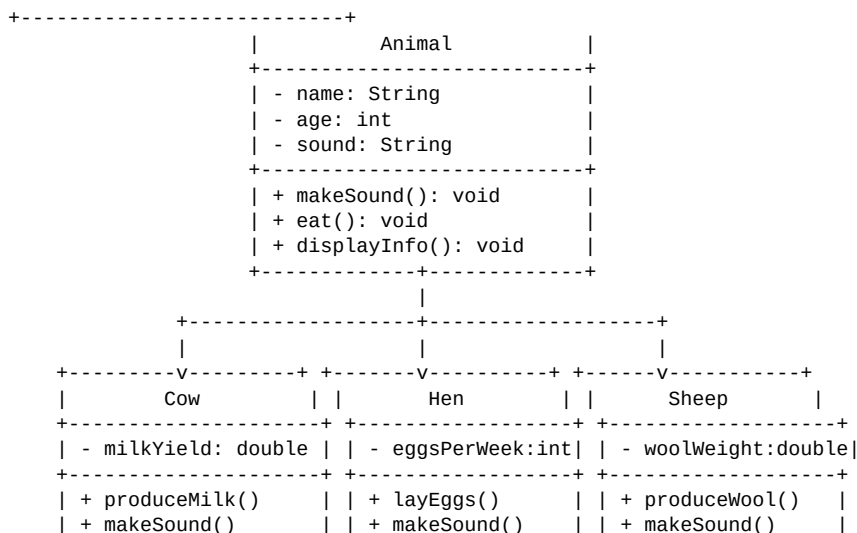
Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

■ Algorithm / Logic Description

1. Create a base class named **Animal**.
2. Define attributes such as: Name, Age, Sound (default noise the animal makes).
3. Create a **Cow** class extending Animal with an additional **milkYield** attribute.
4. Create a **Hen** class extending Animal with an additional **eggsPerWeek** attribute.
5. Create a **Sheep** class extending Animal with an additional **woolWeight** attribute.
6. Define methods in the Animal class for: makeSound(), eat(), displayInfo().
7. Override makeSound() in each subclass to produce the animal-specific sound.
8. Define subclass-specific methods: produceMilk() in Cow, layEggs() in Hen, produceWool() in Sheep.
9. Create objects for each animal type in the main() method.
10. Call makeSound(), eat(), and each animal's specific method to demonstrate polymorphism.
11. Display all animal details using displayInfo().

■ UML Class Diagram



+-----+ +-----+ +-----+

Relationship: Cow, Hen, and Sheep inherit from the Animal class.

■ Source Code

```
class Animal {
    protected String name;
    protected int age;
    protected String sound;

    // Constructor
    Animal(String name, int age, String sound) {
        this.name = name;
        this.age = age;
        this.sound = sound;
    }

    // Default sound
    public void makeSound() {
        System.out.println(name + " makes a sound: " + sound);
    }

    // Eating behavior
    public void eat() {
        System.out.println(name + " is eating.");
    }

    // Display animal info
    public void displayInfo() {
        System.out.println("Name : " + name);
        System.out.println("Age : " + age + " years");
    }
}

// Cow class
class Cow extends Animal {
    private double milkYield; // litres per day

    Cow(String name, int age, double milkYield) {
        super(name, age, "Moo");
        this.milkYield = milkYield;
    }

    @Override
    public void makeSound() {
        System.out.println(name + " says: Moo!");
    }

    public void produceMilk() {
        System.out.println(name + " produces " + milkYield + " litres of milk today.");
    }

    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Milk Yield : " + milkYield + " L/day");
    }
}

// Hen class
class Hen extends Animal {
    private int eggsPerWeek;

    Hen(String name, int age, int eggsPerWeek) {
        super(name, age, "Cluck");
        this.eggsPerWeek = eggsPerWeek;
    }

    @Override
    public void makeSound() {
        System.out.println(name + " says: Cluck cluck!");
    }

    public void layEggs() {
        System.out.println(name + " lays " + eggsPerWeek + " eggs this week.");
    }

    @Override
```

```

    public void displayInfo() {
        super.displayInfo();
        System.out.println("Eggs/Week : " + eggsPerWeek);
    }
}

// Sheep class
class Sheep extends Animal {
    private double woolWeight; // kg per shearing

    Sheep(String name, int age, double woolWeight) {
        super(name, age, "Baa");
        this.woolWeight = woolWeight;
    }

    @Override
    public void makeSound() {
        System.out.println(name + " says: Baa!");
    }

    public void produceWool() {
        System.out.println(name + " produces " + woolWeight + " kg of wool.");
    }

    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Wool Weight: " + woolWeight + " kg");
    }
}

// Main class
public class FarmAnimalsApp {
    public static void main(String[] args) {

        // Creating objects
        Cow cow = new Cow("Bessie", 4, 12.5);
        Hen hen = new Hen("Clucky", 1, 6);
        Sheep sheep = new Sheep("Dolly", 3, 3.2);

        System.out.println("===== COW =====");
        cow.displayInfo();
        cow.makeSound();
        cow.eat();
        cow.produceMilk();

        System.out.println("\n===== HEN =====");
        hen.displayInfo();
        hen.makeSound();
        hen.eat();
        hen.layEggs();

        System.out.println("\n===== SHEEP =====");
        sheep.displayInfo();
        sheep.makeSound();
        sheep.eat();
        sheep.produceWool();

        System.out.println("\n===== POLYMORPHISM DEMO =====");
        Animal[] farmAnimals = { cow, hen, sheep };
        for (Animal a : farmAnimals) {
            a.makeSound(); // calls overridden version for each object
        }
    }
}

```

■ Compilation & Execution Commands

Save the source code exactly as:

FarmAnimalsApp.java

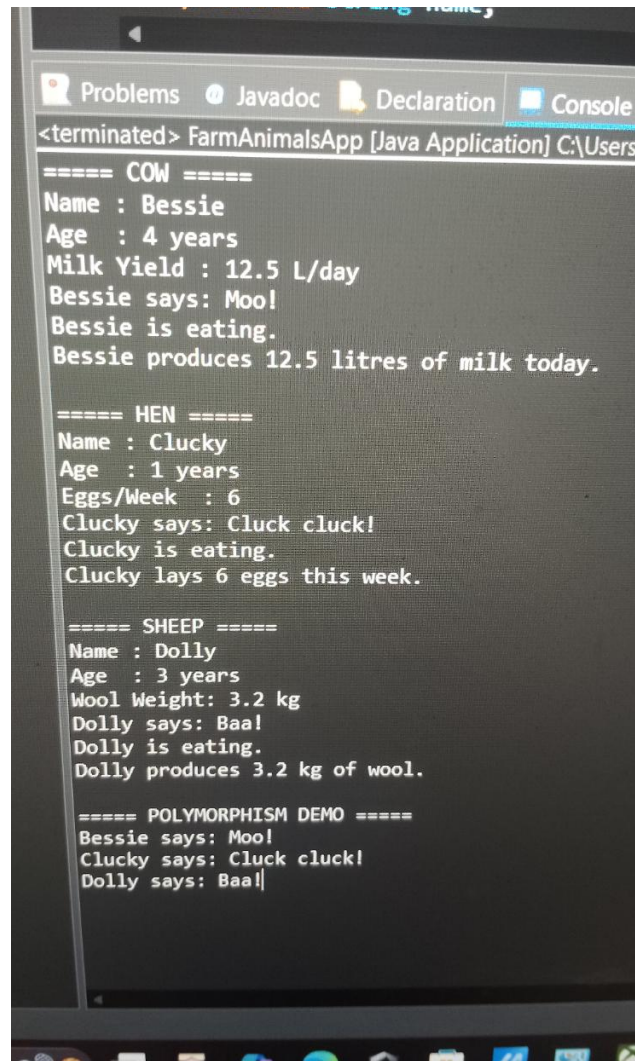
Compile the program:

```
javac FarmAnimalsApp.java
```

Run the compiled program:

```
java FarmAnimalsApp
```

■ Output Screen



```
<terminated> FarmAnimalsApp [Java Application] C:\Users\
===== COW =====
Name : Bessie
Age : 4 years
Milk Yield : 12.5 L/day
Bessie says: Moo!
Bessie is eating.
Bessie produces 12.5 litres of milk today.

===== HEN =====
Name : Clucky
Age : 1 years
Eggs/Week : 6
Clucky says: Cluck cluck!
Clucky is eating.
Clucky lays 6 eggs this week.

===== SHEEP =====
Name : Dolly
Age : 3 years
Wool Weight: 3.2 kg
Dolly says: Baa!
Dolly is eating.
Dolly produces 3.2 kg of wool.

===== POLYMORPHISM DEMO =====
Bessie says: Moo!
Clucky says: Cluck cluck!
Dolly says: Baa!
```

Figure: Console output of FarmAnimalsApp showing animal details, specific methods, and polymorphism demonstration.

■ Result

The Farm Animals App was successfully implemented in Java using classes and objects. The program demonstrates inheritance, method overriding, and runtime polymorphism through Cow, Hen, and Sheep objects derived from a common Animal base class.